Astana IT University

REPORT

Smart Home Environmental Monitoring System

Design and implementation of an indoor monitoring system using ESP32, MQTT protocol, and ThingsBoard cloud visualization.

Student: Assyrzhanova Gaukhar Dulatovna

Group: IIoT-2401

Astana, 2026

Abstract

This report details the design and implementation of an indoor environmental monitoring system prototype, conceptualized for smart home automation. The system is built upon an ESP32 microcontroller and integrates four primary sensors: a DHT22 (temperature and humidity), a PIR sensor (motion), a photoresistor (illuminance), and an MQ-series gas sensor (air quality). Telemetry data is continuously gathered and transmitted via the lightweight MQTT protocol to the ThingsBoard cloud platform for real-time visualization. Furthermore, the system incorporates an automated alerting mechanism that leverages a REST API to send push notifications via a Telegram Bot when hazardous gas concentrations are detected. An analysis of the system's power consumption and theoretical measurement errors confirms the viability of the prototype for practical building automation applications.

Keywords: IoT, ESP32, MQTT, ThingsBoard, smart home, building automation, sensors.

Introduction

The rapid evolution of the Internet of Things (IoT) has fundamentally transformed building automation, offering unprecedented control over indoor environments (Yessenov, 2026b). A modern smart home is no longer just a physical structure; it is a complex ecosystem of interconnected sensors and actuators working in unison to ensure security, comfort, and energy efficiency (Batra & Goyal, 2020).

The primary objective of this project is to construct a functional prototype that bridges the gap between physical environmental variables and cloud-based data analytics. This involves studying the operational principles of common environmental sensors, interfacing them with an edge computing device (ESP32), and establishing a fault-tolerant data transmission pipeline to a cloud dashboard for real-time monitoring and alerting.

System Architecture Overview

The implemented project strictly adheres to the standard three-layer IoT architecture model (Yessenov, 2026a):

1. **Perception Layer:** The physical hardware components, including the ESP32 microcontroller and the array of sensors responsible for data acquisition.

2. **Network Layer:** The utilization of the ESP32's built-in Wi-Fi module combined with the MQTT (Message Queuing Telemetry Transport) protocol to transmit telemetry data.

3. **Application Layer:** The ThingsBoard cloud platform, which serves as the central hub for data processing, graphical visualization, and the execution of alerting rule chains.

Component Analysis and Specifications

The selection of hardware components was driven by the accuracy and energy efficiency requirements outlined in the project parameters.

- **ESP32 DevKit V4:** Selected as the central processing unit due to its dual-core architecture, integrated Wi-Fi and Bluetooth capabilities, and a robust set of General Purpose Input/Output (GPIO) pins with Analog-to-Digital Converter (ADC) support (Batra & Goyal, 2020).

- **DHT22 (AM2302):** A digital sensor utilizing a proprietary single-wire interface. According to its official datasheet, it provides a temperature measurement range of -40°C to 80°C with an accuracy of ±0.5°C, and a humidity range of 0 to 100% RH.

- **PIR Motion Sensor:** Operates based on the pyroelectric effect, detecting changes in infrared radiation emitted by moving objects (Yessenov, 2026b). It outputs a strict discrete logic signal (0 or 1).

- **Photoresistor (LDR):** An analog sensor whose internal resistance decreases as the ambient light intensity increases. It is connected to the ESP32's 12-bit ADC (GPIO 34) to map raw voltage into an estimated lux scale.

- **MQ-Series Gas Sensor:** Configured to utilize its digital output (DOUT) pin to provide instantaneous detection when combustible gas concentrations exceed a safe threshold, a critical requirement for life-safety systems.

Software Development Methodology

The device firmware was developed in C++ utilizing the Arduino IDE framework. The software architecture prioritizes data transmission reliability. The system executes a sensor polling cycle every 5,000 milliseconds (5 seconds), representing an optimal balance between data freshness and network bandwidth conservation.

To facilitate cloud communication, the system employs MQTT, a lightweight publish-subscribe network protocol that runs over TCP/IP (Batra & Goyal, 2020). This significantly minimizes the data overhead compared to traditional HTTP polling, making it ideal for constrained IoT edge devices.

Error Handling and Data Integrity

The firmware incorporates logical safeguards to ensure data integrity. Prior to transmitting temperature and humidity values, the `isnan()` function validates the sensor output. If the DHT22 returns a read error, the system retains and transmits the last known valid state, preventing anomalous data spikes or cloud dashboard failures.

Theoretical Error Calculation

The theoretical measurement errors of the system are derived directly from the hardware datasheets:

1. **Temperature:** ±0.5°C.

2. **Humidity:** ±5% RH.

3. **Illuminance:** The ESP32's internal ADC exhibits known non-linear characteristics at the extremes of its voltage range (near 0V and 3.3V). The linear software mapping introduces an estimated ±8% digitization error, which remains acceptable for relative indoor brightness monitoring.

Power Consumption Analysis

To evaluate the system's operational autonomy, a peak current consumption estimate was conducted (assuming a 3.3V operating logic):

- **ESP32 (Active Wi-Fi Tx):** ~240 mA
- **MQ Gas Sensor (Heater Element):** ~150 mA
- **DHT22 and PIR (Active state):** ~1.6 mA
- **Total Peak Current:** ~391.6 mA

Given the continuous high-current draw of the MQ sensor's heating element, the system consumes approximately 1.3 Watts of power. Consequently, battery operation is impractical, necessitating a stable 5V/1A mains-connected power supply.

Cloud Dashboard and Alerting System

During simulated testing using the Wokwi environment, the ESP32 successfully established a Wi-Fi connection and published JSON payloads to the ThingsBoard platform.

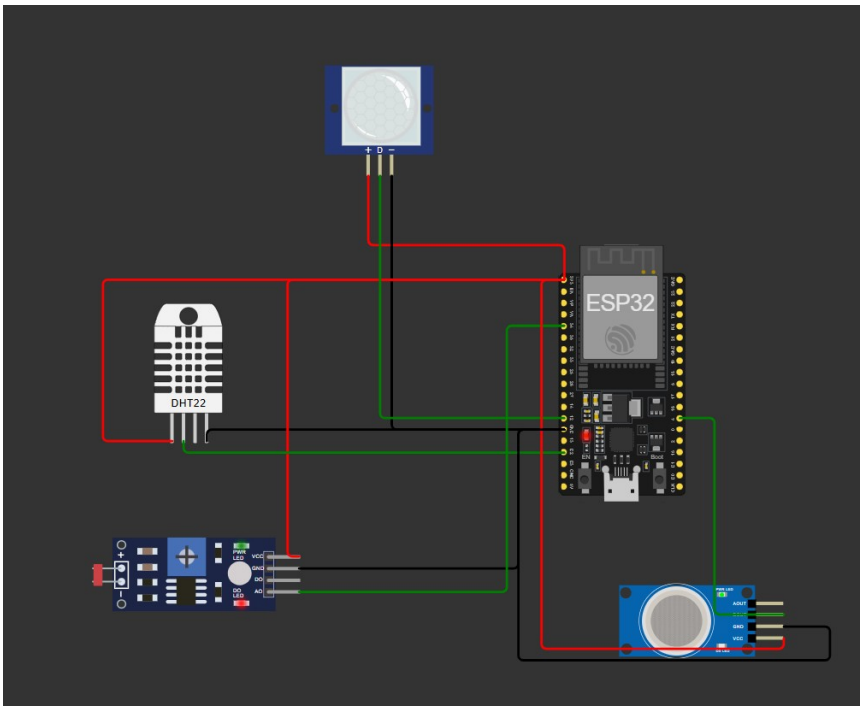

Figure 1. ESP32 hardware wiring diagram implemented in the Wokwi simulator.

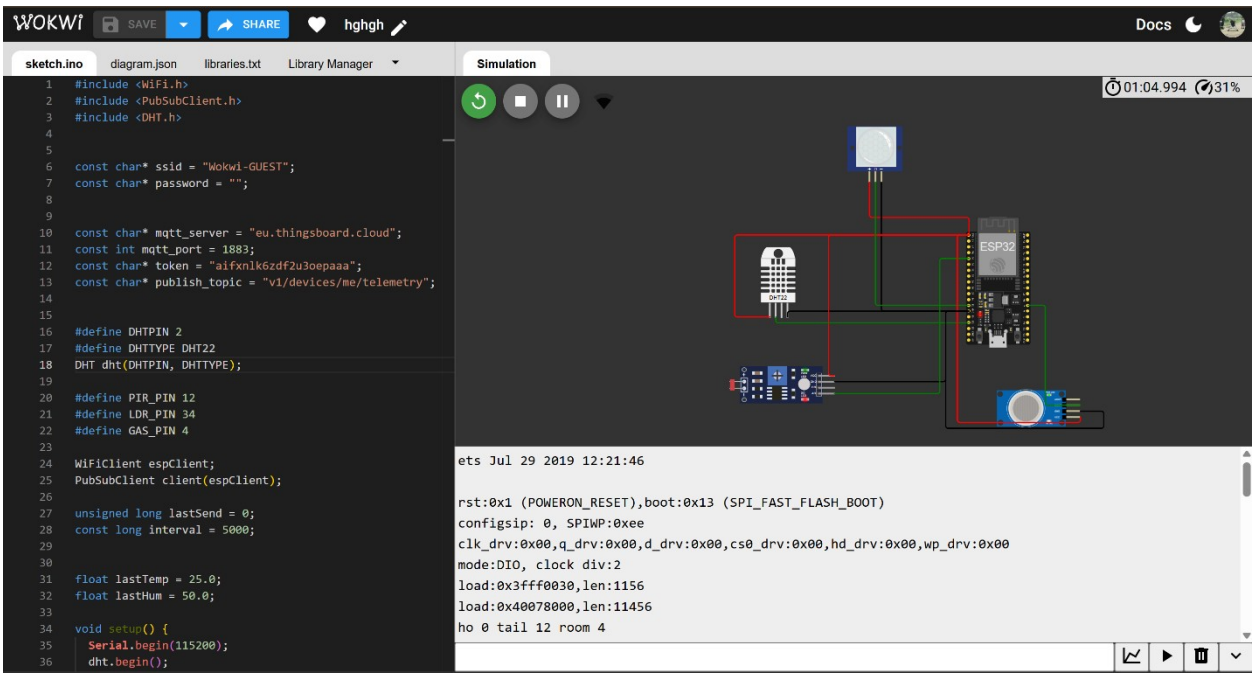

Figure 2. Wokwi simulator full screen

7.1. Virtual Prototyping Approach Before physical assembly, the system was developed and validated using the **Wokwi Online Simulator**. This approach allowed for rapid iterative testing of the circuit logic without the risk of damaging hardware components. The simulator provides a high-fidelity environment for the ESP32 microcontroller, enabling the execution of real-time C++ code and the emulation of Wi-Fi connectivity via the Wokwi-GUEST network.

7.2. Hardware Configuration in Simulation The circuit diagram (see **Figure 1**) illustrates the interconnection of the Perception Layer components:

- **Power Rail:** The ESP32 and all peripheral sensors share a common 3.3V power line and ground (GND) to ensure stable logic levels.
- **Sensor Interfacing:** The **DHT22** is connected to GPIO 2, utilizing a pull-up resistor (internal/external) to stabilize the digital signal.
    - The **PIR sensor** and **MQ-series gas sensor** are interfaced with digital pins (GPIO 12 and GPIO 4 respectively) for binary event detection.

- The **LDR (Photoresistor)** is integrated into a voltage divider circuit connected to GPIO 34, utilizing the ESP32's 12-bit Analog-to-Digital Converter (ADC).

7.3. Simulation Results and Debugging During the simulation phase, the serial monitor was used to debug the MQTT connection sequence. The virtual ESP32 successfully performed the "Handshake" with the eu.thingsboard.cloud server. The simulation environment allowed for manual triggering of sensor states (e.g., sliding the temperature bar or toggling motion), which directly corresponded to the real-time changes observed on the ThingsBoard dashboard. This verified the integrity of the firmware logic and the network transmission layer.

7.4. Verification of Data Transmission (Serial Log)

The successful execution of the firmware and active data transmission is confirmed by the **Serial Monitor output** within the Wokwi interface (see **Figure 3**). As shown in the console log, the ESP32 successfully formats the sensor readings into a JSON payload and transmits them via the MQTT protocol. Each "Sending..." message indicates a successful push to the ThingsBoard telemetry topic. The full implementation logic, including the Wi-Fi handshake and MQTT client configuration, is documented in **Appendix B**.

The ThingsBoard dashboard was populated with custom widgets, including digital gauges for temperature and humidity, time-series charts for illuminance, and boolean status indicators for motion detection.

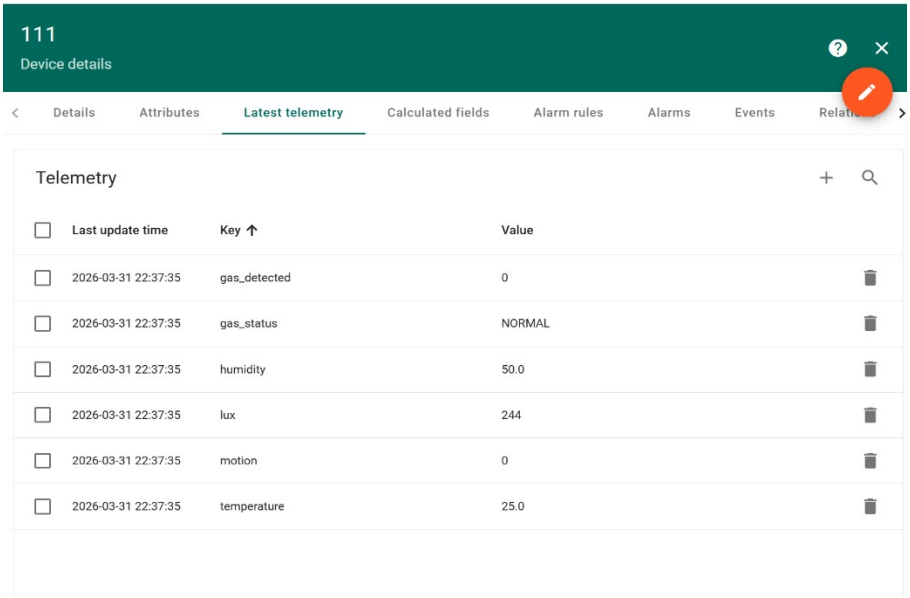

Figure 2. Real-time telemetry data visualization on the ThingsBoard cloud platform.

The dashboard (see Figure 2) provides a real-time visualization of the telemetry data. It includes gauges for ambient temperature and humidity, a time-series chart for illuminance monitoring, and status indicators for occupancy and gas safety. This confirms the successful end-to-end integration of the Perception, Network, and Application layers.

The alerting mechanism was successfully implemented via ThingsBoard Rule Chains. Upon receiving the payload "gas_status": "ALERT", the Rule Engine triggers a REST API POST request directed at the Telegram Bot API. This integration ensures the user immediately receives a push notification on their mobile device stating: "ALERT: Gas detected in the Smart Home!"

Conclusion

The designed prototype successfully fulfills the requirements of the individual project. It practically demonstrates the integration of varied environmental sensors, the implementation of the MQTT protocol, and the utilization of cloud platforms for real-time monitoring and alerting in building automation scenarios. For future iterations, it is highly recommended to implement a

Deep Sleep routine for the ESP32 and transition to low-power MEMS-based gas sensors to enable long-term, battery-powered operation.

References

Batra, N., & Goyal, S. (2020). *IoT fundamentals with a practical approach*. CRC Press.

Espressif Systems. (2023). *ESP32 Series datasheet*. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Yessenov, A. S. (2026a). *Sensors for IoT: Lecture 1 - IoT architecture, protocols and platforms* [Lecture notes]. Astana IT University.

Yessenov, A. S. (2026b). *IoT sensors for smart homes and offices* [Lecture notes]. Astana IT University.

Appendix A: Source Code

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>


const char* ssid = "Wokwi-GUEST";
const char* password = "";


const char* mqtt_server = "eu.thingsboard.cloud";
const int mqtt_port = 1883;
const char* token = "aifxnlk6zdf2u3oepaaa";
const char* publish_topic = "v1/devices/me/telemetry";


#define DHTPIN 2
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

#define PIR_PIN 12
#define LDR_PIN 34
#define GAS_PIN 4

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastSend = 0;
const long interval = 5000;
```

```
float lastTemp = 25.0;
float lastHum = 50.0;

void setup() {
  Serial.begin(115200);
  dht.begin();



  pinMode(PIR_PIN, INPUT);
  pinMode(LDR_PIN, INPUT);
  pinMode(GAS_PIN, INPUT);

  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
}

void setup_wifi() {
  Serial.println("Подключение к Wi-Fi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWi-Fi подключен!");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Подключение к ThingsBoard...");



    if (client.connect("ESP32_Client", token, "")) {
      Serial.println(" OK");
    } else {
      Serial.print("Ошибка ");
      Serial.print(client.state());
      Serial.println(" retry через 3 секунды...");
```

```
      delay(3000);
    }
  }
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  unsigned long now = millis();

  if (now - lastSend > interval) {
    lastSend = now;



    float t = dht.readTemperature();
    float h = dht.readHumidity();



    if (!isnan(t) && t > -40 && t < 80) lastTemp = t;
    if (!isnan(h) && h >= 0 && h <= 100) lastHum = h;

    int presence = digitalRead(PIR_PIN);



    int ldrRaw = analogRead(LDR_PIN);

    int lux = map(ldrRaw, 0, 4095, 0, 1000);



    int gasRaw = digitalRead(GAS_PIN);
    String gasStatus = gasRaw ? "ALERT" : "NORMAL";



    String payload = "{";
    payload += "\"temperature\":" + String(lastTemp, 1) + ",";
    payload += "\"humidity\":" + String(lastHum, 1) + ",";
```

```
        payload += "\"lux\":" + String(lux) + ",";
        payload += "\"gas_detected\":" + String(gasRaw) + ",";
        payload += "\"gas_status\":\"" + gasStatus + "\",";
        payload += "\"motion\":" + String(presence);
        payload += "}";



        Serial.println("Отправка:");
        Serial.println(payload);



        client.publish(publish_topic, payload.c_str());
      }
    }
```

Appendix B: additional photos

```
ets Jul 29 2019 12:21:46

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Подключение к Wi-Fi...
.....
Wi-Fi подключен!
Подключение к ThingsBoard... OK
Отправка:
{"temperature":25.0,"humidity":50.0,"lux":244,"gas_detected":0,"gas_status":"NORMAL","motion":0}
Подключение к ThingsBoard... OK
Отправка:
{"temperature":25.0,"humidity":50.0,"lux":244,"gas_detected":0,"gas_status":"NORMAL","motion":0}
```